# Assessment 3: Algorithm Design Task — Scenario Brief

| | |
|---|---|
| **Module** | UI107006 — Principles of Programming |
| **Assessment** | Algorithm Design Task (25%) |
| **Release Date** | Week 4 |
| **Submission Deadline** | Week 6 |
| **Learning Outcomes** | LO2 (Business), LO4 (Security), LO5 (Technical) |
| **Submission Format** | PDF or DOCX containing diagrams and written explanation |

**Reminder**

You are designing an algorithm — you do not need to write executable code. Your submission should contain flowcharts, structured pseudocode, data type justifications, a security analysis, and a business context discussion. Refer to the Assessment 3 brief for full grading criteria.

# Scenario: Highland Medical Centre — Prescription Verification System

## Background

Highland Medical Centre is a busy GP surgery serving a rural community in the Scottish Highlands. The practice has 6 GPs, 4 nurses, and 2 pharmacists. They currently handle prescription requests through a mix of phone calls, paper forms, and an outdated internal system. The practice manager has commissioned a new Prescription Verification System to improve patient safety and reduce errors.

The system will be used by reception staff to enter prescription requests, and by pharmacists to verify and approve them before dispensing. It must apply a set of clinical safety rules automatically, flagging any issues for human review rather than silently approving potentially dangerous prescriptions.

## Business Context

The practice has identified the following problems with their current process:

- Prescription errors have increased by 15% over the past year, primarily due to manual data entry mistakes and missed drug interaction checks.
- Repeat prescriptions are sometimes issued without verifying that the patient has attended their required review appointment, which is a regulatory requirement.

- Reception staff occasionally process prescriptions for patients who have been flagged as inactive or who have transferred to another surgery, because there is no automated check.
- The existing system has no audit trail, meaning the practice cannot demonstrate compliance during inspections.

The practice manager emphasises that this system handles sensitive medical data and that patient safety is the absolute priority. A prescription that incorrectly passes verification could result in serious harm. The system must err on the side of caution — it is always better to flag a prescription for human review than to approve it incorrectly.

## Your Task

**Design an algorithm** for the prescription verification process. Your algorithm must take a prescription request as input, apply the business rules described below, and produce one of three outcomes: **APPROVED**, **FLAGGED FOR REVIEW**, or **REJECTED**.

## Input Data

Each prescription request contains the following information:

| Field | Data Type | Description |
|---|---|---|
| patient_id | String | Unique patient identifier (e.g., "HC-20451") |
| patient_name | String | Full name of the patient |
| date_of_birth | Date | Patient's date of birth |
| patient_status | String | One of: "active", "inactive", "transferred" |
| medication_name | String | Name of the prescribed medication |
| dosage | String | Dosage amount and frequency (e.g., "500mg twice daily") |
| prescribing_gp | String | Name of the GP issuing the prescription |
| is_repeat | Boolean | True if this is a repeat prescription |
| last_review_date | Date | Date of the patient's most recent review appointment (may be empty) |
| current_medications | List of Strings | All medications the patient is currently taking |
| known_allergies | List of Strings | Patient's recorded allergies (may be empty) |
| request_date | Date | The date the prescription was requested |

## Reference Data Available to Your Algorithm

Your algorithm may also look up information from the following reference sources. You do not need to design these data stores — assume they exist and your algorithm can query them.

## Drug Interaction List

A list of known dangerous drug combinations. Each entry contains two medication names that must not be prescribed together. For example:

| Medication A | Medication B | Severity |
|---|---|---|
| Warfarin | Ibuprofen | High — increased bleeding risk |
| Metformin | Contrast Dye | High — risk of lactic acidosis |
| Simvastatin | Erythromycin | Medium — increased statin toxicity risk |
| Lisinopril | Potassium Supplements | Medium — risk of hyperkalaemia |

## Allergy-Medication Cross-Reference

A list mapping known allergies to medications that should not be prescribed. For example, a patient with a recorded penicillin allergy should not be prescribed Amoxicillin.

## Repeat Prescription Policy

The practice policy states that repeat prescriptions may only be issued if the patient has attended a review appointment within the last 6 months. If the last_review_date is more than 6 months before the request_date, or if it is empty, the repeat prescription must be flagged for review.

## Business Rules

Your algorithm must apply the following rules in order. Note that a prescription may trigger multiple rules — your algorithm should collect all applicable flags and determine the final outcome based on the most severe result.

| Rule | Condition | Outcome | Reason |
|---|---|---|---|
| **R1** | patient_status is "transferred" or "inactive" | **REJECTED** | The patient is no longer registered at this practice. |
| **R2** | medication_name appears in the patient's known_allergies cross-reference | **REJECTED** | Dispensing this medication could cause an allergic reaction. |
| **R3** | medication_name has a HIGH severity interaction with any medication in current_medications | **REJECTED** | High-severity drug interactions pose an immediate danger. |
| **R4** | medication_name has a MEDIUM severity interaction with any medication in current_medications | **FLAGGED FOR REVIEW** | Medium-severity interactions require pharmacist judgement. |
| **R5** | is_repeat is true AND last_review_date is empty or more than 6 months before request_date | **FLAGGED FOR REVIEW** | Repeat prescriptions require a recent review appointment. |
| **R6** | The patient's age (calculated from date_of_birth and request_date) is under 16 or over 80 | **FLAGGED FOR REVIEW** | Prescriptions for young or elderly patients require additional pharmacist verification. |
| **R7** | None of the above rules are triggered | **APPROVED** | The prescription passes all automated checks. |

**Outcome Priority**
If multiple rules are triggered, the most severe outcome takes precedence: REJECTED > FLAGGED FOR REVIEW > APPROVED. Your algorithm should not stop at the first triggered rule — it should evaluate all rules so that the audit log captures every issue found.

## Expected Output

For each prescription processed, your algorithm must produce a verification result containing:

- The final outcome: APPROVED, FLAGGED FOR REVIEW, or REJECTED.
- A list of all rules that were triggered (may be empty if APPROVED).
- A human-readable summary message explaining the decision (e.g., "Rejected: patient has a recorded allergy to Penicillin and Amoxicillin is a Penicillin-derivative").
- A timestamp of when the verification was performed.
- The identity of the user who submitted the request (for audit purposes).

## Test Scenarios

Use the following scenarios to verify your algorithm logic. Your flowcharts or pseudocode should be able to handle all of these correctly.

### Scenario A: Straightforward Approval

- Patient: Jane Murray (HC-10234), active, age 45, no allergies
- Medication requested: Amoxicillin 500mg, three times daily
- Current medications: none
- Not a repeat prescription
- **Expected outcome:** APPROVED — no rules triggered.

### Scenario B: Allergy Rejection

- Patient: Robert Innes (HC-20891), active, age 62, allergy: Penicillin
- Medication requested: Amoxicillin 250mg, twice daily
- Current medications: Metformin
- Not a repeat prescription
- **Expected outcome:** REJECTED — Rule R2 (Amoxicillin is a Penicillin-derivative, cross-referenced against Penicillin allergy).

### Scenario C: Drug Interaction Flag

- Patient: Mary Campbell (HC-15567), active, age 71, no allergies
- Medication requested: Simvastatin 40mg, once daily
- Current medications: Erythromycin, Lisinopril
- Not a repeat prescription
- **Expected outcome:** FLAGGED FOR REVIEW — Rule R4 (Simvastatin + Erythromycin is a medium-severity interaction).

### Scenario D: Multiple Rules Triggered

- Patient: Angus Fraser (HC-30102), active, age 82, allergy: none
- Medication requested: Ibuprofen 400mg, three times daily
- Current medications: Warfarin
- Repeat prescription: yes, last review date: 14 months ago
- **Expected outcome:** REJECTED — Rule R3 (Ibuprofen + Warfarin is high-severity), also Rule R5 (overdue review), and Rule R6 (age over 80). The most severe outcome (REJECTED) applies, but all three rules should be logged.

### Scenario E: Transferred Patient

- Patient: Fiona MacDonald (HC-08913), transferred, age 34, no allergies
- Medication requested: Omeprazole 20mg, once daily
- Current medications: none
- Not a repeat prescription

- **Expected outcome:** REJECTED — Rule R1 (patient status is "transferred").

## Security Considerations

Your security analysis (worth 20% of the grade) should identify at least two security risks inherent in this scenario and explain how your algorithm design mitigates them. To help you think about this, consider the following questions:

- This system handles patient medical records. What are the implications under data protection regulations? Who should be able to access which data?
- Prescription data is entered by reception staff via a user interface. What could go wrong with the input? How could malicious or accidental bad input affect the system?
- The system produces an audit log of all verification decisions. Who should be able to read or modify this log? What would happen if someone could tamper with it?
- The drug interaction and allergy reference data must be accurate for the system to work safely. What happens if this data is outdated or corrupted?
- Multiple staff members will use the system simultaneously. Are there risks around concurrent access?

> **Note**
> You are not expected to design a complete security architecture. Choose at least two risks that are most relevant to your algorithm design and discuss them in enough depth to show understanding. Explain what could go wrong and what your design does (or recommends) to prevent it.

## Business Context Discussion

Your business context discussion (worth 10% of the grade, 200–300 words) should address how your design demonstrates:

- **Ethics:** The system handles life-affecting decisions. How does your algorithm prioritise patient safety? How does it ensure fairness and avoid bias?
- **Timeliness:** Reception staff need to process prescriptions quickly. How does your algorithm balance thoroughness with speed? What happens if the reference data lookup is slow?
- **Quality:** How does your design ensure accuracy? How does the audit trail support quality assurance and regulatory compliance?

## Submission Checklist

Before submitting, confirm your document includes all of the following:

| Requirement | LO |
|---|---|
| Problem statement in your own words showing understanding of the business context | LO2 |
| One or more flowcharts showing overall algorithm logic with decision points, loops, and data flow | LO5 |

| | |
|---|---|
| Structured pseudocode or step-by-step outline supplementing your flowcharts | LO5 |
| Explanation of data types and structures chosen, with justification | LO5 |
| Security analysis identifying at least two risks with mitigation strategies | LO4 |
| Business context discussion (200–300 words) on ethics, timeliness, and quality | LO2 |

## Flowchart Notation

You may use any standard flowchart notation. If you are unsure, the following shapes are conventional:

- Rounded rectangle (terminal): START and END points.
- Rectangle (process): An action or calculation step.
- Diamond (decision): A yes/no or true/false branching point.
- Parallelogram (input/output): Reading data in or producing output.
- Arrow (flow): Direction of execution.

You may create flowcharts using any tool: draw.io (free), Lucidchart, Microsoft Visio, or even neat hand-drawn diagrams that are scanned and included in your document. Ensure they are legible.